BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

LESSON 020 · Stacks and queues with arrays

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does scores[2] access in a C# array?

Q2. What is the difference between an array and a List<T> in C#?

Q3. What does the start pointer identify in a linked list?

Q4. For data = [10, 20, 30, 40], next = [2, -1, 1, 0] and start = 3, list the data values visited in order by following next pointers until -1.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — stack

Recall the term — queue

Recall the term — LIFO

Recall the term — FIFO

Recall the term — push

Recall the term — pop

Recall the term — enqueue

New learning

Stack array — CODE:
int[] stack = new int[10];
int top = -1; — -1 means the stack is empty.

Push and pop — CODE:
top++;
stack[top] = value;

value = stack[top];
top--; — Push moves top up; pop reads then moves top down.

Queue array — CODE:
int[] queue = new int[10];
int front = 0;
int rear = -1; — rear tracks the newest item and front tracks the oldest.

Enqueue and dequeue — CODE:
rear++;
queue[rear] = value;

value = queue[front];
front++; — For this introductory linear queue, pointers move forward through the array.

Worked example

Trace a stack

1. Store 8 — CODE:
top = 0

stack[0] = 8

2. Store 7 — CODE:
top = 1
stack[1] = 7

3. Remove two values — CODE:
first = 7
second = 8
top = -1

4. Store result — CODE:
top = 0
stack[0] = 15

We Do · Practise with guidance

Trace a stack

OCR uses a calculator-style stack with push/pop operations.

Complete the same state

```
Start: empty
push 8 → [8]
push 7 → [?]
A: pop two, add, push → [?]
```

```
Start: empty
push 8 → [8]
push 7 → [8, 7]
A: pop two, add, push → [15]
```

Stack vs queue

Describe one difference.

Complete the comparison

```
Stack = ?
Queue = ?
```

```
Stack = LIFO
Queue = FIFO
```

Push code

Complete array-based push.

Fill the same code

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

```
top = ?;
stack[top] = value;

top = top + 1;
stack[top] = value;
```

Dequeue code

Complete array-based dequeue.

```
value = queue[?];
front = ?;
```

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Implement push/pop
Create an integer stack array and write code that pushes 5, 8 and 3, then pops and prints one item.

Question 2

Implement enqueue/dequeue
Create an integer queue and enqueue 10, 20, 30. Dequeue and print one item.

Question 3

Compare order
Use the same values 2, 4, 6 in a stack and queue and print the first removed value from each.

Question 4

Explain an error
Using the OCR 2023 calculator-stack context, explain what happens if an operation tries to pop more values than are stored.

Exit Ticket · Answer before checking

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

A stack removes the newest stored item first, while a queue removes the oldest stored item first.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. The third element, because array indexing starts at zero.

Q2. An array has a fixed length after creation; a List<T> can grow and shrink.

Q3. The location or index of the first node in the logical sequence.

Q4. 40 → 10 → 30 → 20, then -1 ends the traversal.

You Do pages · Independent questions

Q1. CODE:

```
int[] stack = new int[5];
int top = -1;
top++; stack[top] = 5;
top++; stack[top] = 8;
top++; stack[top] = 3;
Console.WriteLine(stack[top]); // 3
top--;
Console.WriteLine(stack[top]); // 8
```

The top pointer becomes 1, pointing to 8.

Q2. CODE:

```
int[] queue = new int[5];
int front = 0;
int rear = -1;
rear++; queue[rear] = 10;
rear++; queue[rear] = 20;
rear++; queue[rear] = 30;
Console.WriteLine(queue[front]); // 10
front++;
```

The front pointer becomes 1, pointing to 20.

Q3. CODE:

```
int[] stack = { 2, 4, 6 };
int top = 2;
Console.WriteLine(stack[top]); // 6
```

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

```
top--;
int[] queue = { 2, 4, 6 };
int front = 0;
Console.WriteLine(queue[front]); // 2
front++;
```

Stack: last in, first out (6). Queue: first in, first out (2).

Q4. Stack underflow occurs when a pop operation is attempted on an empty stack (top = -1). The program should check whether the stack is empty before reading an array element. Otherwise it may access an invalid index or produce an error.

Exit Ticket · Model answer

True. A stack is last-in first-out; a queue is first-in first-out.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: